

JavaScript Key Learnings — Question 7

OJT Assessment Prep — Polyfill, Array.prototype, .call(), Sparse Arrays

1. What is a Polyfill?

A **polyfill** is code you write to replicate a built-in method for environments that don't support it — like legacy smart-TV browsers.

```
// Old browser missing .filter()?
// Write your own version that works the same way!
Array.prototype.myFilter = function() { ... }

// Now every array has .myFilter() even in old browsers
```

■ **Rule:** A polyfill must behave exactly like the original method — same arguments, same output.

2. Array.prototype — Adding Methods to ALL Arrays

When you add a method to **Array.prototype**, every array in your code gets that method automatically.

```
Array.prototype.sayHello = function() {
  console.log("I have " + this.length + " items");
};

[1, 2, 3].sayHello();    // → "I have 3 items"
["a", "b"].sayHello();  // → "I have 2 items"

// Inside the method, "this" = the array it was called on
```

3. How .filter() Works — Before Writing the Polyfill

The real **.filter()** loops through an array and keeps only items where the callback returns **true**.

```
const nums = [1, 2, 3, 4, 5];

const result = nums.filter(function(item) {
  return item > 2; // keep only items where this is true
});
// result → [3, 4, 5]

// Callback receives: (currentItem, index, wholeArray)
```

4. thisArg — The Optional Context Parameter

The real **.filter()** accepts a second argument — **thisArg** — which sets what **this** is inside the callback.

```
const checker = { maxPrice: 50 };

[20, 150, 30].filter(function(item) {
```

```
    return item < this.maxPrice; // this = checker
  }, checker);                 // thisArg = checker
// → [20, 30]
```

Your polyfill must support *thisArg* too — passed via *.call()*.

5. Sparse Arrays — Skipping Empty Slots

A **sparse array** has holes — empty slots that are not even undefined:

```
const sparse = [1, , 3]; // index 1 is a HOLE

// "in" checks if an index actually EXISTS
0 in sparse // → true  (has value 1)
1 in sparse // → false (empty hole!)
2 in sparse // → true  (has value 3)

// In the polyfill – skip holes:
if (!(i in this)) continue;
```

6. .call() — Running a Function with Custom this

.call(thisArg, arg1, arg2) calls a function and lets you manually set what **this** is.

```
function greet() {
  console.log(this.name);
}
const person = { name: "John" };

greet();           // this = window → undefined
greet.call(person); // this = person → "John"
```

In the polyfill:

```
callback.call(thisArg, this[i], i, this)
//           ↑       ↑   ↑   ↑
//           set this  current idx whole
//           to thisArg item      array
```

7. The Complete Polyfill — Line by Line

```
Array.prototype.myFilter = function(callback, thisArg) {
  let result = []; // empty basket for results

  for (let i = 0; i < this.length; i++) { // visit every index

    if (!(i in this)) continue; // skip empty holes in sparse arrays

    if (callback.call( // run the test function
      thisArg,         // set "this" inside callback
      this[i],         // arg1: current item
```

```

        i,                // arg2: current index
        this              // arg3: whole array
    )) {
        result.push(this[i]);    // test passed → keep the item
    }
}

return result;                // return collected items
};

```

8. Visual Flow — How It All Connects

Item	Price	price < 50?	Result
Mug	\$20	true	■ kept
Keyboard	\$150	false	■ removed
Notebook	\$30	true	■ kept
Mouse	\$80	false	■ removed

■ ■ Complete Solution

```

// 1. The Polyfill
Array.prototype.myFilter = function(callback, thisArg) {
    let result = [];
    for (let i = 0; i < this.length; i++) {
        if (!(i in this)) continue;
        if (callback.call(thisArg, this[i], i, this)) {
            result.push(this[i]);
        }
    }
    return result;
};

// 2. Products data
const products = [
    { id: 1, price: 20, name: "Mug" },
    { id: 2, price: 150, name: "Keyboard" },
    { id: 3, price: 30, name: "Notebook" },
    { id: 4, price: 80, name: "Mouse" },
];

// 3. Filter under $50
const affordable = products.myFilter(function(product) {
    return product.price < 50;
});

// 4. Render to DOM

```

```
const main = document.getElementById("storefront");
affordable.forEach(function(data) {
  const article = document.createElement("article");
  const h2 = document.createElement("h2");
  h2.textContent = data.name;
  const p = document.createElement("p");
  p.textContent = "$" + data.price;
  article.appendChild(h2);
  article.appendChild(p);
  main.appendChild(article);
});
```

■ Golden Rules to Remember

Concept	Rule
Polyfill	Replicate built-in methods for old browsers — same args, same output
Array.prototype	Adding here gives ALL arrays the method; this = the calling array
thisArg	Optional 2nd arg to filter() — sets this inside callback via .call()
Sparse arrays	Use "i in this" to skip empty holes before calling callback
.call()	Runs a function with a manually set this context